RSCL — Algorithm 2: Symbolic Emergency Channel and emergency_key() Normalisation

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

hu-telepresence-gazebo/festival/hu_live_server.py	sha256 2416370e93dc	2026-11-11 06:38 UTC	clean @ 10ee643
hu-telepresence-gazebo/festival/control_server_hu.py	sha256 3f32578a144c	2026-11-11 06:32 UTC	clean @ 10ee643
hu-telepresence-gazebo/festival/hu_live_dashboard.html	sha256 d1e2cf6aabae	2026-11-12 01:23 UTC	clean @ 05172be

1. Purpose and context

An operator must be able to stop or land the whole fleet without depending on a neural model, a confidence score, an aircraft name or a confirmation dialog. The symbolic emergency channel recognises a small closed set of emergency phrases (English and Arabic-derived *khalas*, *yalla*) after deterministic normalisation, and dispatches a fleet-wide hold or landing with a `symbolic_emergency` flag that the supervisor accepts without the model thresholds of Algorithm 1. It is checked **first** for every utterance, before mission routing and before the model.

2. Inputs, outputs, data

Item	Definition
Input	Utterance after voice normalisation (Algorithm 19 alias table, e.g. *yella/yellow/yala → yalla*).
EMERGENCY_LAND	{land now, land land land, everybody down, everyone down, emergency land, yalla land}
EMERGENCY_HOVER	{stop, stop stop stop, all stop, everyone stop, everybody stop, khalas, abort}
Output	Decision record with band EMERGENCY, action HOVER or LAND, targets = all five names, confidence 1.0, `decision_source = SYMBOLIC SAFETY (NOT TRAINED HU)`; dispatch `emergency_hover` / `emergency_land`.

3. Algorithm

Algorithm 2.1 emergency_key (phrase canonicalisation)

Source: festival/hu_live_server.py: emergency_key

```
function EMERGENCY-KEY(text)
  s ← lower(text); replace every non-word, non-space character by ' '; collapse whitespace
  PRE(x)  ← strip leading  "(hey|ok|okay )? (all|everyone|everybody|all drones|drones|team|guys|fleet) "
  SUF(x)  ← strip trailing " (please|right now|immediately)"
  NOW(x)  ← strip trailing " now"
  for cand in [ s, PRE(s), SUF(s), PRE(SUF(s)), SUF(PRE(s)), NOW(PRE(SUF(s))), NOW(SUF(s)) ] do
    if cand ∈ EMERGENCY_LAND ∪ EMERGENCY_HOVER then return cand
  return None
```

Algorithm 2.2 Emergency dispatch

Source: festival/hu_live_server.py: Service.command, Service.voice_auto; festival/control_server_hu.py: /api/hu/dispatch, Fleet.command

```
procedure EMERGENCY(u_norm, source)
  k ← EMERGENCY-KEY(u_norm) or lower(u_norm).strip(" .!?")
  if k ∉ EMERGENCY then return NOT-EMERGENCY           // continue with Alg. 3
  act ← emergency_hover if k ∈ EMERGENCY_HOVER else emergency_land
```

```
  POST /api/hu/dispatch {act, targets=ALL, confidence=1.0, symbolic_emergency=true} with token
  record band EMERGENCY, dispatched ← (response.ok = true)
// supervisor side
procedure SUPERVISOR-EMERGENCY(act)
  token check; confidence thresholds skipped because symbolic_emergency ∧ act ∈ {emergency_*}
  targets ← ALL;  action ← land (emergency_land) or hover (emergency_hover)
  stale ← aircraft without fresh odometry (age ≥ 2.0 s)
  targets ← targets \ stale; log SAFETY_SKIPPED for stale        // safe-direction: do not block the
fleet
  if targets = ∅ then refuse
  emergency_hover bypasses the fault block; land is never fault-blocked
  for d in targets do
    if action = hover then
      QUEUED → GROUNDED (cancel staggered launch); parked/FAULT → hold z; else HOVER at max(0.25, z)
    else                                                          // land
      QUEUED → GROUNDED; parked → unchanged; else fault ← false, target ← (x, y, 0.12), mode ← LANDING
```

Voice: in `voice_auto`, if the alias-corrected transcript is an emergency phrase, it is executed immediately (no spoken confirmation of the correction), whereas any other corrected transcript requires a separate confirmation (Algorithm 19). Typed input is never held for review.

4. Flowchart and state diagram

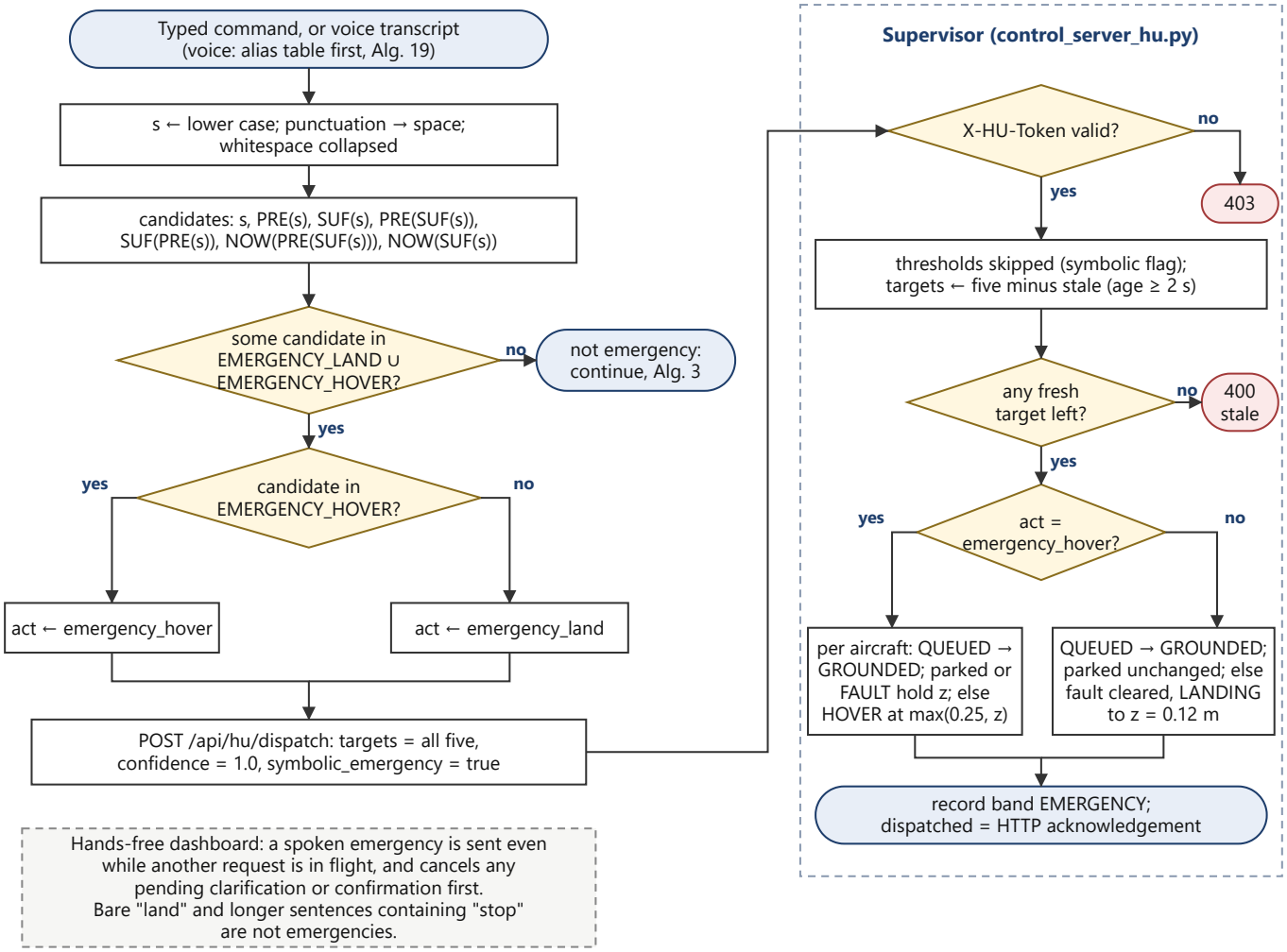


Figure 2.1. Flowchart of the symbolic emergency channel (`emergency_key`, `Service.command`, `/api/hu/dispatch`, `Fleet.command`). Stale aircraft are skipped and logged as `SAFETY_SKIPPED`; the request fails only if no target is fresh.

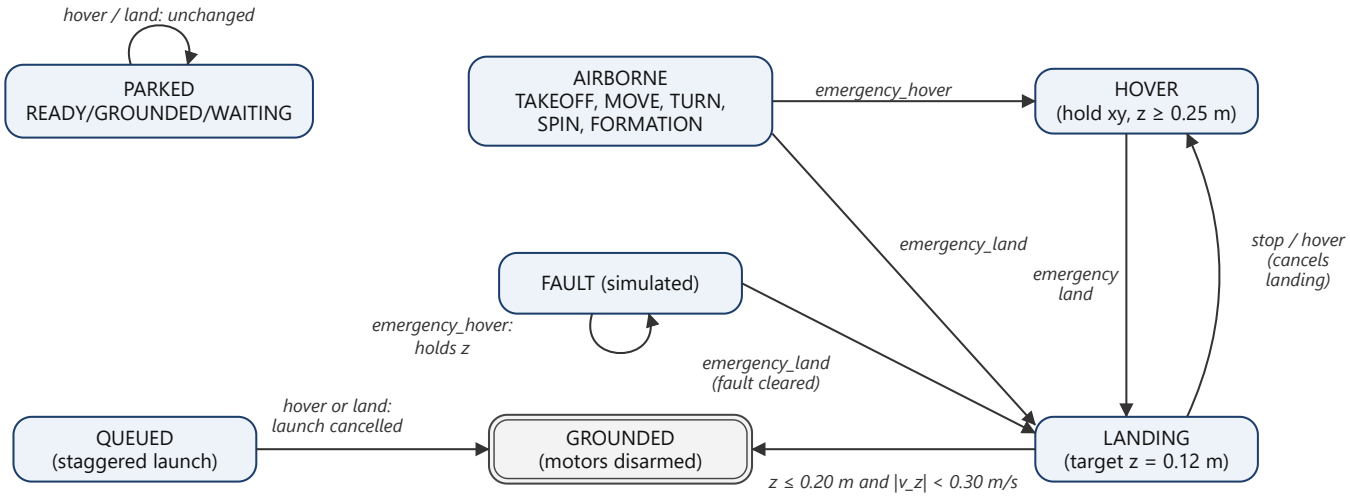


Figure 2.2. The channel itself is stateless (it keeps no memory between utterances); the state it acts on is the per-aircraft mode in the supervisor (`Fleet.command`, `Fleet.loop`). The diagram shows only transitions caused by the emergency commands and the touchdown rule. Note the LANDING → HOVER edge: a later "stop" turns an emergency landing into a hover, because hover is applied to every non-parked, non-faulted mode. Aircraft with stale odometry receive no command at all.

5. Parameters

Parameter	Value
Phrase sets	6 landing phrases, 7 hold phrases (above); bare *land* is deliberately excluded
Confidence sent	1.0 (a constant marker, not a model output)
Telemetry freshness	odometry age < 2.0 s (`Fleet.fresh`)
Landing target / touchdown	z = 0.12 m; disarm when z ≤ 0.20 m and $\|v_z\| < 0.30$ m/s; descent rate limited to [−0.32, +0.10] m/s
Hover floor	0.25 m

6. Complexity

EMERGENCY-KEY performs at most 7 set look-ups on strings of length ≤ 300: O(|u|). Dispatch is O(N) in the number of aircraft (N = 5). No model inference; the live fault scenario recorded an acknowledgement of 3.0 ms (see Section 8).

7. Properties

Guaranteed

- Independence from the neural model: recognition is exact set membership after deterministic rewriting; no score is involved.
- Precedence: the check runs before mission routing and model inference for every typed and voice command.
- Bare *land*, and any phrase not in the closed set after rewriting, is *not* an emergency.
- Safe-direction availability: one aircraft with stale odometry does not block landing or holding the others; the skipped aircraft are logged (`SAFETY_SKIPPED`).
- The runtime proximity watchdog never cancels a landing (LANDING is excluded from proximity holds, Algorithm 7), so close aircraft are not frozen in the air during an emergency landing.
- A land/hold never launches a parked aircraft and cancels queued staggered launches.

Not guaranteed

- Stale aircraft receive no new command (the control loop sends zero velocity with motors not re-enabled; Algorithm 9); the channel cannot land an aircraft whose telemetry is lost.
- Recognition depends on the speech recogniser producing a phrase in the set (plus the alias table); paraphrases such as "please put them all down" are not recognised and fall through to Algorithms 3/1.
- The phrase *stop* embedded in a longer sentence is not an emergency (only whole-utterance matches after the listed rewrites).
- An emergency landing is not latched: a later *stop* / *hover* (emergency or not) sets every aircraft that is still LANDING back to HOVER (Figure 2.2), because hover applies to all non-parked, non-faulted modes.
- Landing is vertical at the current XY; no landing-site deconfliction is performed.
- Emergency dispatch does not wait for physical confirmation; `dispatched` reflects the HTTP acknowledgement only.

8. Testing and measured results

- `acceptance/tests/test_emergency_phrases.py`: hover variants, land variants, non-emergency phrases (incl. bare *land*). `acceptance/tests/test_http_v9.py::test_symbolic_hover_authorized_without_model_threshold`. `test_v9.py::test_stop_is_hover_not_landing`, `test_bare_land_does_not_land_whole_fleet`.
- Live: `acceptance/fault_test.py`, run `run_20261111T052914Z`: spoken ASR variant "yella land" with one aircraft faulted → band EMERGENCY, ack 3.0 ms, landing began at 0.0 s, all five grounded at 6.39 s, nobody re-launched (PASS).

9. Comparison with alternatives and related work

The channel is a small, trusted path that can override the complex (neural and planning) command path. That is the idea behind the Simplex architecture and run-time assurance (RTA); the industrial counterpart is a hardware emergency stop.

- **Simplex architecture** [1, 2]: a verified, simple safety controller runs beside a complex controller; a decision module switches to the safety controller when the plant approaches the boundary of a recoverable region. The switch is automatic and based on the plant state.
- **Run-time assurance standard ASTM F3269-21** [3]: specifies RTA components (monitor, recovery function, switch) and requirements for bounding the behaviour of unassured complex functions in aircraft systems, with assurance levels from a system safety assessment.
- **SOTER** [4]: a programming framework in which each RTA module pairs an advanced and a safe controller with a safety specification; well-formed modules carry a formal guarantee. It was evaluated on drone software in simulation and on real vehicles.
- **Hardware emergency stop** (ISO 13850:2015) [5]: a dedicated, operator-initiated stop function on machinery, with design principles independent of the energy type; for electrical equipment the realisation is in IEC 60204-1.

Approach	Trigger	Independent of the complex path?	Assurance	This channel, by comparison
Simplex [1, 2]	automatic, from plant state	yes (separate controller and switch)	safety controller verified	operator-triggered only; automatic protection is separate (Alg. 7); no verification
ASTM F3269 RTA [3]	monitor of a safety boundary	required by architecture	design assurance level from safety assessment	not designed to the standard; no assurance level

SOTER [4]	monitor per module, time-bounded switching	yes	formal guarantee for well-formed modules	ad hoc code; tested, not proved
Hardware e-stop [5]	physical operator action	yes, hard-wired	functional-safety standards	software command over HTTP in the same process chain as normal commands
This document	closed phrase set after deterministic rewriting	independent of the model and thresholds; shares bridge, token, network and supervisor	unit and live tests	—

Where this channel is weaker. It is not independent in the sense of [1]–[5]: it depends on the speech recogniser (for voice), the browser, the HU bridge process, the token file, the local HTTP link and the supervisor process, and it cannot reach an aircraft whose odometry is stale. It does not monitor anything by itself. A real-vehicle deployment would need a separate stop path (e.g. a hard-wired or radio kill switch) that does not pass through any of this software. **Where it is useful.** Within the simulation it gives the operator a model-free override with a measured 3.0 ms acknowledgement, and it removes the confidence threshold from the safe-direction action, as RTA recovery functions do.

10. Relation to other algorithms

First stage of the command pipeline; on no match, control passes to Algorithm 3 (routing) and possibly Algorithm 1. Uses the supervisor primitives and stale-skipping rule of Algorithm 9 and the landing exemption of Algorithm 7. Algorithm 11 (coordinator) uses the same `emergency_hover` path as its escalation when a normal hold is refused.

11. References

1. D. Seto, B. Krogh, L. Sha and A. Chutinan, "The Simplex architecture for safe on-line control system upgrades," *Proc. American Control Conference*, Philadelphia, 1998, pp. 3504–3508. `https://doi.org/10.1109/ACC.1998.703255`
2. L. Sha, "Using Simplicity to Control Complexity," *IEEE Software*, 18(4):20–28, 2001. `https://doi.org/10.1109/MS.2001.936213`
3. ASTM F3269-21, *Standard Practice for Methods to Safely Bound Behavior of Aircraft Systems Containing Complex Functions Using Run-Time Assurance*, ASTM International, 2021. `https://store.astm.org/f3269-21.html`
4. A. Desai, S. Ghosh, S. A. Seshia, N. Shankar and A. Tiwari, "SOTER: A Runtime Assurance Framework for Programming Safe Robotics Systems," *IEEE/IFIP DSN* 2019, pp. 138–150. `https://arxiv.org/abs/1808.07921`
5. ISO 13850:2015, *Safety of machinery — Emergency stop function — Principles for design*, 3rd ed., International Organization for Standardization, 2015. Catalogue entry: `https://www.sis.se/en/produkter/environment-health-protection-safety/safety-of-machinery/iso138502015/`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.